

Drone Flight Control Firmware

Sample Requirements (HAL, RTOS, Autopilot) and C Coding Guidelines

This document provides a starter template for requirements management and a pragmatic C coding standard for embedded flight control firmware. It is aimed at single-team projects building PX4-grade software (not formally certified to DO-178C) while borrowing the most valuable discipline from that standard: bidirectional traceability and a disciplined coding subset. Adapt freely to your project.

1. Requirements Structure

Requirements are organised in three levels and three module domains. Every requirement has a unique ID, a single testable statement, a rationale, a parent (the higher-level requirement it satisfies), and a defined verification method.

Levels

SYS — System-level requirements. What the vehicle must do from the operator's point of view. Cross-module.

HLR — High-Level software Requirements. What a software module must do to satisfy SYS requirements.

LLR — Low-Level software Requirements. How a module is internally structured (timing, algorithms, data flow).

Module prefixes

HAL-* — Hardware Abstraction Layer (drivers, MCU peripherals).

RTOS-* — Real-Time Operating System (scheduler, IPC, timing).

AP-* — Autopilot (estimation, control, navigation, failsafes).

SYS-* — System-wide requirements that span modules.

ID format

```
MOD-SUB-NNN
  MOD : module      (HAL | RTOS | AP | SYS)
  SUB : sub-area    (IMU, SCHED, ATT, NAV, SAFE, ...)
  NNN : 3-digit number, never reused after deletion
```

Examples: HAL-IMU-001, RTOS-SCHED-004, AP-ATT-012, SYS-SAFE-002

Verification methods

Test — automated unit, integration, or HIL test.

Analysis — static analysis, formal proof, or hand calculation.

Inspection — code review against a checklist.

Demonstration — observed behaviour in a defined scenario (typically flight test).

2. System Requirements (SYS)

These are vehicle-level requirements. They are the parents of HLRs in HAL, RTOS, and Autopilot modules. Shown here as a small representative set.

ID	SYS-SAFE-001
Title	Emergency disarm
Statement	The vehicle shall disarm all motors within 200 ms of receiving a valid kill-switch command from the radio link.
Rationale	Operator must be able to terminate thrust immediately to prevent injury or property damage.
Parent	(top level)
Verification	Test (HIL) + Demonstration (flight test)

ID	SYS-SAFE-002
Title	RC link loss behaviour
Statement	On loss of RC signal for more than 1.0 s, the vehicle shall transition autonomously to Return-To-Launch if GPS fix is valid, otherwise to Land mode.
Rationale	Defined failsafe avoids fly-aways and uncontrolled descent.
Parent	(top level)
Verification	Test (SITL) + Demonstration (flight test)

ID	SYS-NAV-001
Title	Position hold accuracy
Statement	In Position-Hold mode with valid GPS and steady wind up to 8 m/s, the vehicle shall remain within a 2 m radius of the commanded position for at least 60 s.
Rationale	Required for stable camera work and safe operation near obstacles.
Parent	(top level)
Verification	Demonstration (flight test)

ID	SYS-PWR-001
Title	Low-battery RTL
Statement	When battery state-of-charge falls below the configured RTL threshold (default 25%), the vehicle shall automatically initiate Return-To-Launch.
Rationale	Prevents loss of vehicle due to depleted battery during autonomous missions.
Parent	(top level)
Verification	Test (SITL) + Demonstration

ID	SYS-TIM-001
Title	Cold-boot time
Statement	The vehicle shall be ready to arm within 30 s of power-on under nominal sensor conditions.
Rationale	Operational requirement for field deployment.
Parent	(top level)
Verification	Test (bench)

3. HAL Module Requirements

The HAL isolates the firmware from MCU-specific peripherals and bus implementations. It exposes a stable C API and is replaceable per target (real MCU build, host-stub build for unit tests, simulator build).

3.1 High-Level Requirements (HAL-HLR)

ID	HAL-IMU-001
Title	IMU sample rate
Statement	The IMU driver shall deliver synchronised gyroscope and accelerometer samples at a configurable rate, with 1 kHz as the default.
Rationale	Rate-loop control at 1 kHz is required by AP-RATE-001.
Parent	SYS-NAV-001, AP-RATE-001
Verification	Test (target HIL: oscilloscope on DRDY pin and timestamp log)

ID	HAL-IMU-002
Title	Sample validity flag
Statement	Each IMU sample delivered to upper layers shall carry a validity flag that is false if the sample failed CRC, range, or self-test checks.
Rationale	Estimator must reject corrupt samples to avoid divergence.
Parent	SYS-SAFE-002
Verification	Test (unit, fault injection)

ID	HAL-PWM-001
Title	ESC output update rate
Statement	The motor PWM/DSHOT driver shall update all configured outputs synchronously at a rate selectable between 400 Hz and 8 kHz.
Rationale	Synchronous update prevents per-motor command skew that would couple into attitude error.
Parent	AP-RATE-001
Verification	Test (target HIL, logic analyser)

ID	HAL-API-001
Title	Host-stub portability
Statement	The HAL public API shall be implementable on a POSIX host such that all HAL-dependent code above the HAL boundary compiles and runs without modification on the host.
Rationale	Enables host-side unit testing of all upper layers.
Parent	(process)
Verification	Inspection + Test (host build CI job)

ID	HAL-TIME-001
Title	Monotonic time source
Statement	The HAL shall provide a monotonic microsecond counter with drift not exceeding +/-50 ppm over the operating temperature range.
Rationale	Stable time base required by control loops and EKF.
Parent	AP-RATE-001
Verification	Analysis (datasheet) + Test (24h drift measurement)

3.2 Low-Level Requirements (HAL-LLR)

ID	HAL-IMU-101
Title	ICM-42688 SPI configuration
Statement	On initialisation, the IMU driver shall configure the ICM-42688 to 8 kHz internal ODR, 2000 dps gyro range, 16 g accel range, and shall verify WHO_AM_I == 0x47.
Rationale	Concrete sensor configuration realising HAL-IMU-001.
Parent	HAL-IMU-001
Verification	Test (unit, with SPI mock) + Test (target HIL)

ID	HAL-IMU-102
Title	DRDY interrupt handler
Statement	The IMU DRDY interrupt handler shall read one FIFO packet, timestamp it with the HAL monotonic clock, and post it to the IMU queue, completing within 50 microseconds worst case.
Rationale	Bounded ISR duration preserves rate-loop jitter budget.
Parent	HAL-IMU-001, RTOS-ISR-001
Verification	Test (target, WCET measurement)

ID	HAL-PWM-101
Title	DSHOT600 frame timing
Statement	DSHOT600 output frames shall be encoded with a bit period of 1.67 microseconds +/-2%, using DMA-driven timer PWM.
Rationale	Realises HAL-PWM-001 for DSHOT600.
Parent	HAL-PWM-001
Verification	Test (target HIL, logic analyser)

ID	HAL-API-101
Title	Header-only API
Statement	All HAL public symbols shall be declared in hal/*.h with no transitive include of MCU vendor headers.
Rationale	Prevents vendor types leaking into upper layers.
Parent	HAL-API-001
Verification	Analysis (grep in CI)

4. RTOS Module Requirements

The RTOS provides task scheduling, inter-task communication, and synchronisation primitives. Requirements describe behaviour observable by clients of the RTOS API, not internal implementation.

4.1 High-Level Requirements (RTOS-HLR)

ID	RTOS-SCHED-001
Title	Preemptive priority scheduling
Statement	The scheduler shall implement fixed-priority preemptive scheduling with at least 16 priority levels.
Rationale	Required to guarantee hard real-time deadlines for control loops above lower-priority background tasks.
Parent	AP-RATE-001
Verification	Test (host port + target stress)

ID	RTOS-SCHED-002
Title	Context switch latency
Statement	Worst-case context switch latency on the target MCU shall not exceed 5 microseconds.
Rationale	Bounded context switch time is part of the rate-loop timing budget.
Parent	AP-RATE-001
Verification	Test (target, GPIO-toggle WCET measurement)

ID	RTOS-IPC-001
Title	Message queue API
Statement	The RTOS shall provide a thread-safe, lock-free or bounded-blocking message queue API supporting fixed-size items and configurable depth.
Rationale	Standard mechanism for ISR-to-task data flow (e.g. IMU samples).
Parent	HAL-IMU-102
Verification	Test (unit + target stress with TSan equivalent)

ID	RTOS-MEM-001
Title	No dynamic allocation after init
Statement	After the scheduler is started, no RTOS API call shall perform dynamic memory allocation.
Rationale	Eliminates fragmentation and unbounded latency.
Parent	(process)
Verification	Analysis (heap usage trace) + Inspection

ID	RTOS-ISR-001
Title	ISR-safe API subset
Statement	A defined subset of RTOS APIs shall be safe to call from interrupt context, documented in the API reference, and shall not disable interrupts for more than 5 microseconds.
Rationale	ISRs (e.g. IMU DRDY) need to wake tasks deterministically.
Parent	RTOS-SCHED-002
Verification	Inspection + Test (target)

ID	RTOS-WD-001
Title	Task watchdog
Statement	The RTOS shall provide a per-task software watchdog API that triggers a configurable callback if a task fails to check in within its declared period.
Rationale	Detect stalled or starved critical tasks.
Parent	SYS-SAFE-002
Verification	Test (unit + target fault injection)

4.2 Low-Level Requirements (RTOS-LLR)

ID	RTOS-SCHED-101
Title	Ready-queue data structure
Statement	The ready queue shall be implemented as a bitmap of priority levels plus a per-priority doubly linked list, giving O(1) highest-priority selection.
Rationale	Bounded scheduling cost regardless of task count.
Parent	RTOS-SCHED-002
Verification	Inspection + Analysis

ID	RTOS-IPC-101
Title	Queue post-from-ISR
Statement	The function <code>rtos_queue_post_isr()</code> shall complete in constant time, never block, and indicate whether a context switch is required on ISR exit.
Rationale	Realises RTOS-IPC-001 for the ISR path.
Parent	RTOS-IPC-001, RTOS-ISR-001
Verification	Test (unit) + Analysis (asm review)

ID	RTOS-MEM-101
Title	Static task definition
Statement	Task control blocks and stacks shall be allocated from statically declared storage provided by the caller of <code>rtos_task_create_static()</code> .
Rationale	Realises RTOS-MEM-001.
Parent	RTOS-MEM-001
Verification	Inspection

ID	RTOS-WD-101
Title	Watchdog tick source
Statement	The task watchdog shall be evaluated from the systick ISR at 1 kHz and shall not perform any operation that may block.
Rationale	Deterministic watchdog evaluation.
Parent	RTOS-WD-001
Verification	Inspection + Test

5. Autopilot Module Requirements

The autopilot module covers state estimation, attitude/rate/position control, navigation, and failsafe logic. It depends only on the HAL and RTOS APIs.

5.1 High-Level Requirements (AP-HLR)

ID	AP-RATE-001
Title	Rate-loop frequency
Statement	The body-rate control loop shall execute at 1 kHz with jitter not exceeding +/-100 microseconds.
Rationale	Loop rate sets achievable bandwidth and disturbance rejection for the attitude controller.
Parent	SYS-NAV-001
Verification	Test (target, timestamp log) + Analysis

ID	AP-ATT-001
Title	Attitude hold accuracy
Statement	In Stabilise or Position-Hold modes, commanded roll and pitch shall be tracked within +/-2 degrees in steady wind up to 8 m/s.
Rationale	Realises SYS-NAV-001 at the attitude level.
Parent	SYS-NAV-001
Verification	Demonstration (flight test) + Test (SITL)

ID	AP-EST-001
Title	Attitude estimator convergence
Statement	The attitude estimator shall converge to within 5 degrees of truth within 3 s of initialisation with the vehicle stationary.
Rationale	Bounded startup time per SYS-TIM-001.
Parent	SYS-TIM-001
Verification	Test (SITL, recorded sensor logs)

ID	AP-EST-002
Title	Sensor fault rejection
Statement	Estimators shall reject any sensor sample whose validity flag is false (per HAL-IMU-002) and shall flag the affected state as degraded after 100 ms of continuous rejection.
Rationale	Prevent corrupt sensor data from driving the vehicle.
Parent	SYS-SAFE-002, HAL-IMU-002
Verification	Test (unit, fault injection)

ID	AP-FS-001
Title	RC-loss failsafe
Statement	On RC-loss notification from the RC driver, the autopilot shall transition to RTL within 100 ms if GPS is valid, otherwise to LAND.
Rationale	Realises SYS-SAFE-002.
Parent	SYS-SAFE-002
Verification	Test (SITL)

ID	AP-FS-002
Title	Kill switch latency
Statement	On receipt of a valid kill-switch command, the autopilot shall command zero motor output within 50 ms.
Rationale	Combined with HAL latency, satisfies SYS-SAFE-001's 200 ms budget.
Parent	SYS-SAFE-001
Verification	Test (SITL) + Demonstration

ID	AP-MEM-001
Title	Static allocation
Statement	The autopilot module shall perform no dynamic memory allocation at any time.
Rationale	Bounded memory use; eliminates a class of runtime failures.
Parent	(process)
Verification	Analysis (linker map, sanitizer)

5.2 Low-Level Requirements (AP-LLR)

ID	AP-RATE-101
Title	Rate-loop trigger
Statement	The rate loop shall be triggered by arrival of a valid IMU sample on the IMU queue and shall not poll the clock.
Rationale	Sample-synchronous execution minimises jitter and latency.
Parent	AP-RATE-001, RTOS-IPC-001
Verification	Inspection + Test

ID	AP-ATT-101
Title	Quaternion representation
Statement	Attitude shall be represented internally as a unit quaternion (w,x,y,z) and re-normalised every iteration of the estimator.
Rationale	Avoids gimbal lock and numerical drift.
Parent	AP-ATT-001, AP-EST-001
Verification	Inspection + Test (unit, fuzz)

ID	AP-CTRL-101
Title	Output saturation
Statement	All control outputs (rate, attitude, position loops) shall be clipped to [-1.0, +1.0] before mixing, and integrator anti-windup shall be applied whenever the unclipped output is outside this range.
Rationale	Prevent integrator windup during saturation.
Parent	AP-ATT-001
Verification	Test (unit) + Analysis (CBMC bounded model check)

ID	AP-FS-101
Title	Mode-transition table
Statement	All mode transitions shall be defined in a single static transition table; ad-hoc mode changes from other modules are prohibited.
Rationale	Reviewable and testable failsafe behaviour.
Parent	AP-FS-001
Verification	Inspection + Test (SITL coverage of all transitions)

ID	AP-NUM-101
Title	Float-only arithmetic in control
Statement	Control-loop code shall use single-precision float and shall not depend on double-precision arithmetic, FPU exceptions, or denormal handling.
Rationale	Deterministic execution on Cortex-M FPU; predictable WCET.
Parent	AP-RATE-001
Verification	Analysis (static check) + Inspection

6. Traceability in Code and Tests

Every implementation file and test file references the requirements it implements or verifies. A simple Python script walking the repository produces the trace matrix that pairs requirements to implementers and verifiers. CI fails if any requirement has zero implementers or zero verifiers, or if any tagged ID does not exist.

Tag in source

```
/**
 * Body-rate controller.
 *
 * @implements AP-RATE-001, AP-RATE-101, AP-CTRL-101
 */
void rate_controller_step(const imu_sample_t *imu,
                          const rate_setpoint_t *sp,
                          motor_cmd_t *out)
{
    /* ... */
}
```

Tag in test

```
/* @verifies AP-RATE-001 */
TEST(rate_ctrl, runs_at_1khz_in_sitl)
{
    /* ... */
}

/* @verifies AP-CTRL-101 */
TEST(rate_ctrl, clips_output_to_unit_range)
{
    /* ... */
}
```

7. C Coding Guidelines

These guidelines define a pragmatic subset of C99/C11 for embedded flight-control firmware. They are influenced by MISRA C:2012, CERT C, the Barr Group Embedded C Coding Standard, and the NASA JPL Power of Ten. They are not full MISRA compliance; they target the rules with the highest return on engineering effort.

7.1 Language and toolchain

R1.1 The codebase shall target C11 with GNU extensions disabled in upper layers (HAL and below may use vendor extensions where unavoidable).

R1.2 All translation units shall compile cleanly with: `-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wcast-align -Wpointer-arith -Wstrict-prototypes -Wmissing-prototypes -Wundef -Wfloat-equal -Werror`.

R1.3 Optimisation level for release shall be `-Os` or `-O2`; `-O3` is prohibited without a documented justification.

R1.4 Compiler version and flags shall be pinned in the build system and reproducible.

7.2 File and module structure

R2.1 Each module exposes exactly one public header in `include/<module>/` and may have any number of private headers in its source directory.

R2.2 Public headers must be self-contained (compile when included in isolation) and idempotent (use `#pragma once` or a unique include guard).

R2.3 A public header must not include another module's private header.

R2.4 Layering rule: HAL has no upward dependencies; RTOS depends only on HAL; Autopilot depends only on HAL and RTOS APIs. Reverse dependencies are prohibited.

R2.5 A `.c` file shall not exceed 1000 lines; split otherwise.

7.3 Naming

R3.1 Public symbols use the module prefix: `hal_imu_read()`, `rtos_queue_post()`, `ap_rate_step()`.

R3.2 Static (file-local) symbols have no prefix but use snake_case.

R3.3 Types use the `_t` suffix: `imu_sample_t`, `rate_setpoint_t`.

R3.4 Macros and compile-time constants are UPPER_SNAKE_CASE.

R3.5 Single-letter names are permitted only as loop indices (`i`, `j`, `k`) and small math variables (`x`, `y`, `z`).

7.4 Types

R4.1 Use fixed-width integer types from `<stdint.h>` exclusively: `uint8_t`, `int16_t`, `uint32_t`, etc. Plain `int`, `long`, `short` are prohibited in interfaces and storage.

R4.2 Use `bool` from `<stdbool.h>` for boolean values; do not use integer 0/1.

R4.3 Use `float` in control and estimation code; `double` only where dynamic range demands it, with a comment justifying the use.

R4.4 Enumerations must have an explicit underlying width when stored or transmitted: declare with values that fit a documented type.

R4.5 Avoid bit-fields; use explicit masks and shifts on `uint32_t` for register access.

7.5 Memory and allocation

R5.1 Dynamic allocation (`malloc`, `calloc`, `free`) is prohibited after the scheduler is started.

R5.2 If allocation is used during init, it must come from a single statically declared pool whose size is checked at compile time.

R5.3 Stack frames in any task shall be bounded; recursion is prohibited.

R5.4 Variable-length arrays (VLAs) are prohibited.

R5.5 All buffers crossing module boundaries must be passed with an explicit length parameter;

null-terminated strings are restricted to logging.

7.6 Control flow

R6.1 All loops shall have a statically determinable upper bound (JPL Power-of-Ten rule 2). For loops awaiting an external event, use a timeout.

R6.2 `goto` is prohibited except for single-target cleanup at function end (Linux-kernel style).

R6.3 Every switch on an enum shall either cover every value or have a default that asserts or returns an error.

R6.4 `if`, `else`, `for`, `while`, and `do` bodies are always braced, even for a single statement.

R6.5 Functions shall have a single point of return where this improves readability; early returns for guard clauses are permitted.

R6.6 Functions shall be 50 statements or fewer where feasible (JPL Power-of-Ten rule 4).

7.7 Functions and parameters

R7.1 Every function has a prototype before first use, in the appropriate header.

R7.2 Public functions shall validate all input parameters at entry; assertions are used for programming errors, returned error codes for runtime conditions.

R7.3 Pointer parameters shall be marked `const` where the pointee is not modified.

R7.4 Output parameters appear after input parameters in the parameter list.

R7.5 Functions shall return a status code (`typedef enum { ... } status_t`) where failure is possible; the return value shall not be ignored at call site (annotate with `__attribute__((warn_unused_result))`).

7.8 Concurrency and RTOS

R8.1 Shared mutable state shall be protected by exactly one documented mechanism (mutex, queue, atomic). Mixing mechanisms on the same data is prohibited.

R8.2 No RTOS API call may block while holding a mutex; lock scope shall be as small as possible.

R8.3 Critical sections (interrupt disable) shall not exceed 5 microseconds and shall be commented with the worst-case duration.

R8.4 ISRs shall do the minimum work and defer processing to a task via a queue or semaphore. No floating-point operations in ISRs unless the FPU is explicitly saved.

R8.5 `Volatile` is used only for memory-mapped registers and objects modified by ISRs. It is not a substitute for synchronisation.

7.9 Error handling

R9.1 Status codes are checked at every call site that can fail; ignoring an error is a defect.

R9.2 Assertions (`FCS_ASSERT`) are used for invariants that must hold by construction; in release builds they shall transition the vehicle to a safe state (LAND or motor shutdown), not silently continue.

R9.3 Errors are logged with a numeric code and originating subsystem; freeform strings are restricted to debug builds.

R9.4 A defined error code shall not be redefined or reused with a different meaning.

7.10 Preprocessor

R10.1 Function-like macros are avoided when an inline function would suffice. Macros that exist must be parenthesised and use `do { ... } while (0)` when they expand to statements.

R10.2 `#ifdef` for feature selection is restricted to the HAL layer and the build configuration header.

R10.3 Magic numbers in code are prohibited; use named constants (`static const` or `enum`).

7.11 Comments and documentation

R11.1 Every public function has a Doxygen comment describing purpose, parameters, return value, preconditions, and the requirement IDs it implements (@implements).

R11.2 Comments explain *why*, not *what*. Code that needs a 'what' comment should be rewritten or named more clearly.

R11.3 TODO/FIXME comments shall include an issue tracker ID.

R11.4 Commented-out code is prohibited in committed code; delete it (the VCS keeps history).

7.12 Tooling and CI gates

R12.1 CI shall run on every PR and shall fail the build on any of: compiler warnings, clang-tidy violations, cppcheck violations (including MISRA addon), failed unit tests, drop in coverage.

R12.2 Host-side unit-test builds shall enable AddressSanitizer and UndefinedBehaviorSanitizer.

R12.3 A static analysis baseline is maintained; new violations are not allowed, existing ones are tracked and burned down.

R12.4 Coverage reports (gcov/lcov) are published per build; branch coverage on the autopilot core shall not drop below an agreed threshold (e.g. 85%).

R12.5 Every requirement ID referenced in code or tests must exist in the requirements database; CI fails otherwise.

Appendix A — Recommended free tools

Purpose	Tool(s)
Unit testing (C)	Unity + Ceedling + CMock, or cmocka, or Check
Static analysis	clang-tidy, cppcheck (+ MISRA addon), scan-build, Infer
Sanitizers	ASan, UBSan, TSan, MSan (gcc/clang)
Memory checking	Valgrind (host)
Coverage	gcov + lcov + gcovr
Fuzzing	libFuzzer, AFL++
Formal methods	CBMC, Frama-C (EVA, WP), KLEE
MCU emulation	Renode, QEMU
Simulation	Gazebo, jMAVSim, JSBSim, FlightGear
Requirements	Doorstop, StrictDoc, OpenFastTrace
CI	GitHub Actions, GitLab CI

Appendix B — References

- RTCA DO-178C, Software Considerations in Airborne Systems and Equipment Certification
- MISRA C:2012, Guidelines for the Use of the C Language in Critical Systems
- CERT C Coding Standard, SEI Carnegie Mellon
- Barr Group, Embedded C Coding Standard
- NASA JPL, The Power of Ten — Rules for Developing Safety-Critical Code (Holzmann)
- ARP4754A — Guidelines for Development of Civil Aircraft and Systems
- ARP4761 — Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment

This document is a starting template. Adapt requirement IDs, numeric thresholds, and coding rules to your project's actual hardware, safety analysis, and team practices.